



Escola Superior
de Tecnologia e Gestão

Licenciatura em Engenharia Informática

Escola Superior de Tecnologia e Gestão

Instituto Politécnico de Viana do Castelo

Unidade curricular de

OPÇÃO II – APRENDIZAGEM
ORGANIZACIONAL / SISTEMAS DE
SUPORTE À DECISÃO

2025/2026

Meeting Intelligence - Track B (AI Agents) - c25

Cristóvão Lavarinhas nº27629

Tiago Araújo nº31433

Índice

1. Descrição do Projeto	3
2. Arquitetura.....	5
2.1. Estrutura do Repositório	6
2.2. App Streamlit.....	7
2.2.1. Instalação	7
2.2.2. Funcionamento	8
2.3. Extensão Chrome.....	9
2.3.1. Instalação	9
2.3.2. Funcionamento	9
3. Resultados e Validação	11
4. Dificuldades, Resolução e Limitações	13
4.1. Transcrição em tempo real inviável	13
4.2. Restrições de CORS.....	13
4.3. Captura de áudio em Manifest V3.....	13
4.4. Conflito de cuDNN entre pyannote e Whisper.....	14
4.5. Conflito entre bibliotecas de áudio	14
4.6. Eco no microfone.....	14
4.7. Outras Dificuldades	14
5. Conclusão	15
6. Bibliografia	16

Índice de Figuras

Figura 1 - Interface inicial da aplicação Streamlit	3
Figura 2 - Popup da extensão Chrome em estado inativo	4
Figura 3 - Pipeline geral do Meeting Intelligence: captura, diarização, transcrição e análise.....	5
Figura 4 - Estrutura de pastas	6
Figura 5 - terminal a correr streamlit run src/app.py	7
Figura 6 - Resultados da análise apresentados na aplicação Streamlit	8
Figura 7 - Extensão Chrome carregada via Load unpacked	9
Figura 8 - Sidebar da extensão durante a captura de áudio.....	10
Figura 9 - Transcrição diarizada e análise estruturada apresentadas na sidebar, com cores distintas por locutor	11

1. Descrição do Projeto

O Meeting Intelligence é um sistema de inteligência para reuniões que transforma gravações de áudio em conhecimento estruturado. O sistema é composto por duas componentes complementares: uma **aplicação web em Streamlit**, para processar ficheiros de áudio carregados pelo utilizador, e uma **extensão Chrome**, para captura e análise em tempo real de reuniões realizadas no Google Meet.

Ambas as componentes seguem o mesmo pipeline conceptual: o áudio é primeiro **diarizado** (identificação de quem fala e quando), depois **transcrito** (conversão de fala em texto), e por fim **analisado** por um modelo de linguagem que extrai informação estruturada: um resumo da reunião, decisões tomadas, próximos passos com responsável e prazo atribuídos, e questões levantadas que ficaram sem resposta.

O objetivo do projeto é eliminar a necessidade de alguém tomar notas manualmente durante uma reunião, garantindo que toda a informação relevante é capturada e organizada de forma consistente, independentemente de quem estava a prestar atenção no momento.

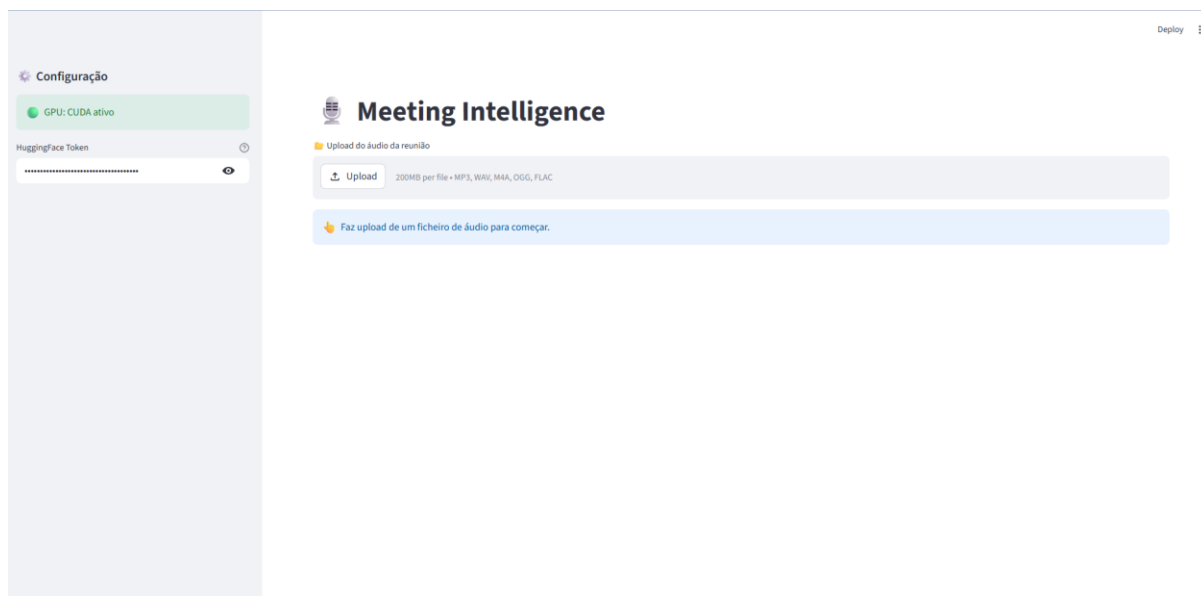


Figura 1 - Interface inicial da aplicação Streamlit

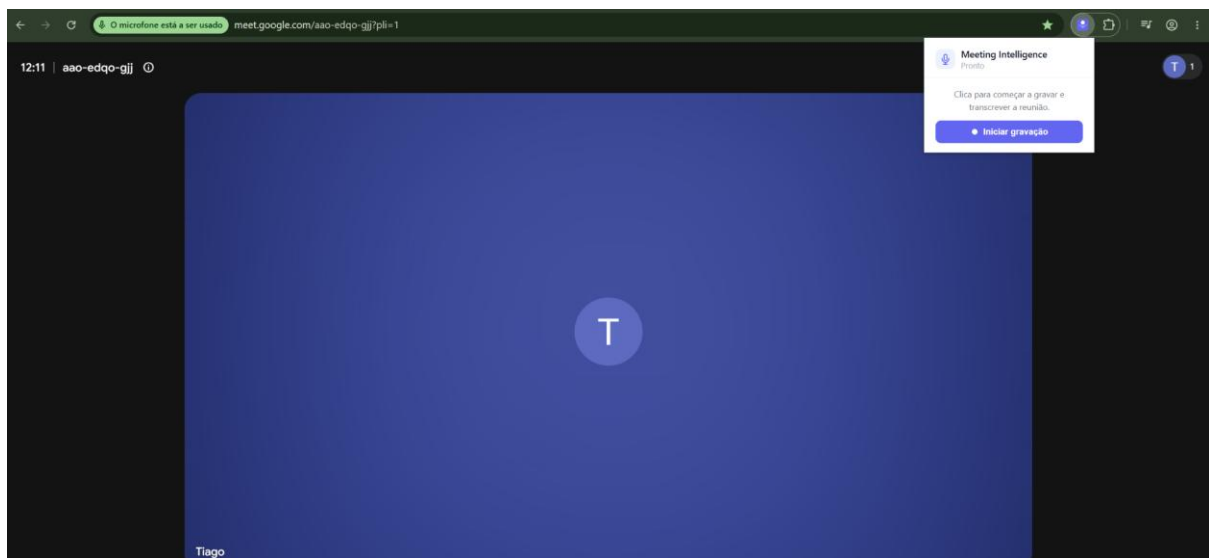


Figura 2 - Popup da extensão Chrome em estado inativo

2. Arquitetura

O sistema assenta num pipeline de quatro etapas: captura de áudio, diarização, transcrição e análise semântica.

A **diarização** é feita com o **pyannote/speaker-diarization-3.1**, um modelo pré-treinado disponibilizado via HuggingFace, que segmenta o áudio em intervalos de tempo atribuídos a diferentes locutores (**SPEAKER_00**, **SPEAKER_01**, etc.), sem identificar quem são as pessoas, apenas que são vozes distintas.

A **transcrição** é feita com o Whisper, modelo de reconhecimento de fala que converte os segmentos de áudio identificados pela diarização em texto.

A **análise semântica** é feita pelo Claude, acedido através da plataforma IAedu. A transcrição completa, já organizada por locutor, é enviada ao modelo com um prompt estruturado que pede explicitamente um JSON com os campos **summary**, **decisions**, **action_items** (com **owner**, **task** e **deadline**) e **open_questions**. Esta escolha foi deliberada: em vez de pedir um resumo em texto livre, força-se o modelo a produzir **dados estruturados e atribuíveis**, cumprindo o requisito de extração estruturada com atribuição de responsáveis.

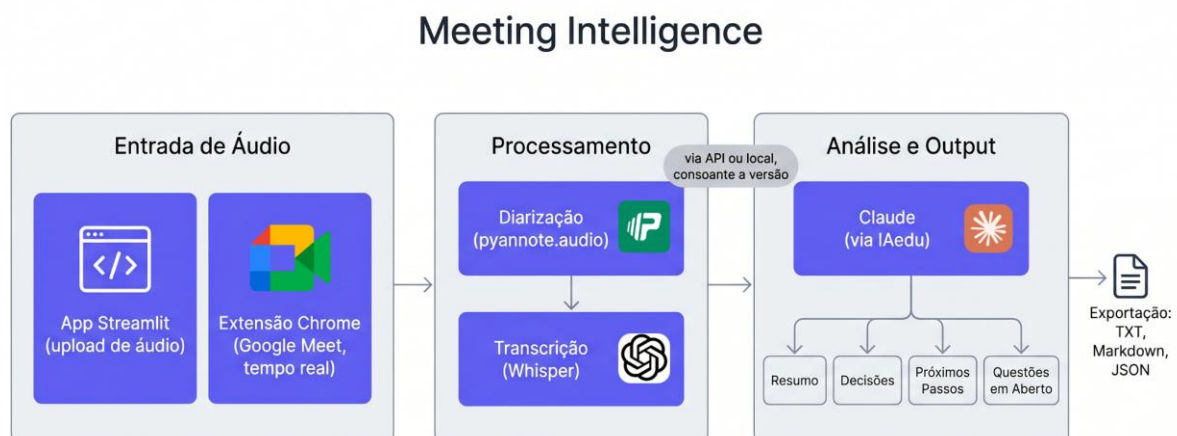


Figura 3 - Pipeline geral do Meeting Intelligence: captura, diarização, transcrição e análise.

2.1. Estrutura do Repositório

O código está organizado num único repositório GitHub, com uma estrutura pensada para separar claramente o código fonte, a documentação e os dados de teste:

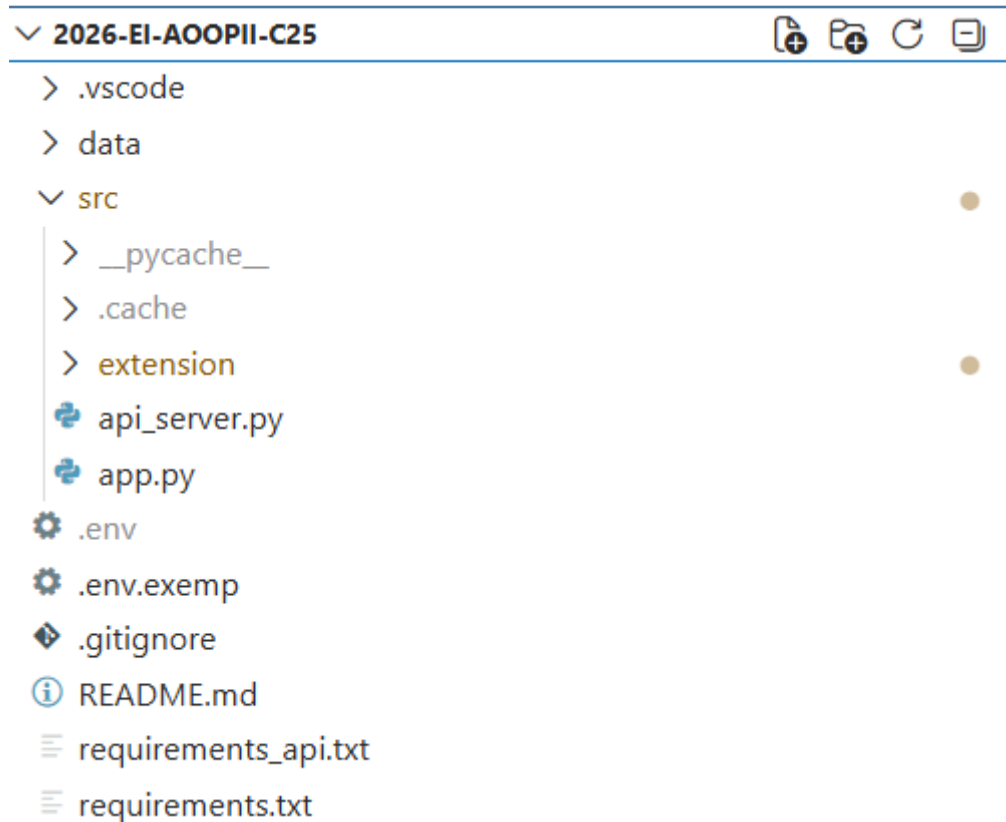


Figura 4 - Estrutura de pastas

A extensão Chrome encontra-se dentro de **src/**, junto com o restante código fonte, mantendo a sua própria estrutura interna por exigência do Chrome (o **manifest.json** e os ficheiros associados têm de estar na raiz da pasta carregada como extensão). O **README.md** documenta a identificação do grupo, a descrição do projeto, o stack tecnológico utilizado e os passos necessários para correr cada uma das componentes.

2.2. App Streamlit

A aplicação Streamlit oferece uma interface web onde o utilizador carrega um ficheiro de áudio. Existe em **duas versões**, pensadas para públicos diferentes: a versão da branch principal (main), de utilização geral, e a versão local, disponível noutra branch.

2.2.1. Instalação

A **versão da branch principal (main)** corre inteiramente através de chamadas a APIs externas (IAedu para transcrição e análise), pelo que **qualquer pessoa pode usá-la apenas configurando as suas API keys**, sem necessidade de hardware específico nem de instalar modelos pesados localmente. Após clonar o repositório e configurar o ficheiro `.env` (a partir do template `.env.exemp`) com o token da HuggingFace e as credenciais do Claude via IAedu, basta instalar as dependências leves (`requirements_api.txt`) e correr `streamlit run src/app.py`.

A **versão local** corre os modelos de diarização e transcrição diretamente na máquina do utilizador, sem depender de keys de transcrição externas. Requer GPU (idealmente NVIDIA com CUDA) para correr com desempenho aceitável. A instalação exige primeiro o PyTorch com suporte para a GPU disponível (CUDA 12.1 ou 11.8, consoante o modelo de placa gráfica), seguido das dependências completas (`requirements.txt`), antes de correr `streamlit run src/app.py`.

Em ambos os casos, é necessário um token válido da HuggingFace, com aceitação prévia dos termos de utilização dos modelos `pyannote/speaker-diarization-3.1` e `pyannote/segmentation-3.0`, sem os quais a diarização não funciona.

```
PS C:\git\2026-ei-aopii-c25\src> & "$env:LOCALAPPDATA\Programs\Python\Python310\python.exe" -m streamlit run app.py
2026-06-24 11:55:48.562 Uvicorn server started on 0.0.0.0:8501

You can now view your Streamlit app in your browser.

Local URL: http://localhost:8501
Network URL: http://192.168.1.157:8501
```

Figura 5 - terminal a correr `streamlit run src/app.py`

2.2.2. Funcionamento

Após o utilizador carregar um ficheiro de áudio, a aplicação corre o pipeline completo de forma sequencial: o áudio é primeiro convertido para um formato uniforme, depois diarizado com o pyannote, transcrito (via IAedu ou Whisper local, consoante a versão), e por fim analisado pelo Claude. No final, a interface apresenta a transcrição organizada por locutor, as métricas da reunião (número de locutores, duração, número de blocos de fala) e a análise estruturada com resumo, decisões, próximos passos e questões em aberto. O utilizador pode exportar os resultados em três formatos: TXT, Markdown e JSON.

The screenshot displays the 'Meeting Intelligence' application interface. On the left is a 'Configuração' sidebar with 'GPU: CUDA ativo' and a 'HuggingFace Token' field. The main area shows a 'Meeting Intelligence' header with an audio upload button and a progress bar at 0:00 / 1:38. Below this, a red bar indicates 'Análise concluída' with a green checkmark. A summary table shows 4 locutors, 1.6 min duration, and 7 blocks of speech. The 'Análise Inteligente' section includes a summary, 4 decisions, 2 next steps, and 0 open questions. The 'Acta da Reunião' section lists speaker turns with their respective text. At the bottom, an 'Exportar' section offers buttons for 'Download TXT', 'Download Markdown', and 'Download JSON'.

Figura 6 - Resultados da análise apresentados na aplicação Streamlit

2.3. Extensão Chrome

A extensão Chrome (Manifest V3) foi desenhada para captura em tempo real durante reuniões no Google Meet.

2.3.1. Instalação

A instalação da extensão é independente da app Streamlit. Primeiro corre-se o servidor Flask local (**api_server.py**), responsável pela transcrição e diarização, com as dependências **flask**, **flask-cors**, **faster-whisper**, **pyannote.audio**, **soundfile** e **torch** instaladas. De seguida, é necessário criar o ficheiro **config.js** dentro da pasta da extensão com as credenciais da IAedu (este ficheiro não é submetido ao repositório, por conter informação sensível). Por fim, a extensão é carregada manualmente no Chrome através da opção **Load unpacked**, em modo de desenvolvedor, apontando para a pasta **src/extension/**.

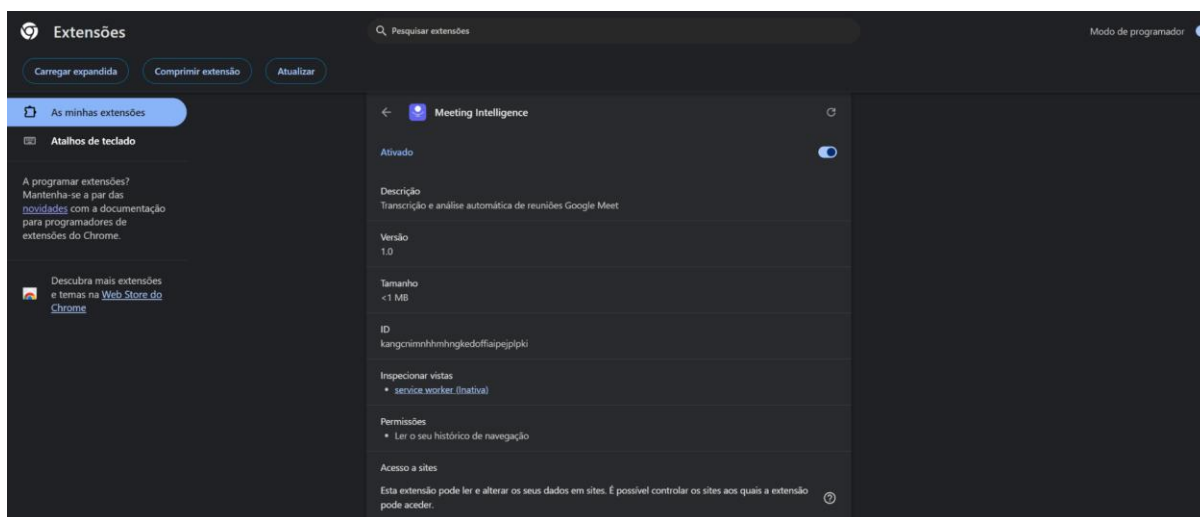


Figura 7 - Extensão Chrome carregada via Load unpacked

2.3.2. Funcionamento

O utilizador inicia a gravação através de um popup, que despoleta dois pedidos de permissão: partilha do separador (**getDisplayMedia**, para captar o áudio dos restantes participantes) e acesso ao microfone (**getUserMedia**, com cancelamento de eco ativo, para captar a voz do utilizador local). Os dois streams são combinados

num único **AudioContext**, garantindo que o áudio de todos os intervenientes, local e remotos, fica num único ficheiro coerente para diarização.

Ao contrário da app Streamlit, a extensão **não consegue usar a IAedu para transcrição**: a API é um agente de texto e não aceita ficheiros de áudio enviados a partir do browser. Para resolver isto, foi criado um **servidor Flask local (api_server.py)**, que corre na máquina do utilizador na porta 5000 e expõe um único endpoint responsável por receber o áudio, correr a diarização com pyannote e a transcrição com Whisper, e devolver os blocos de texto organizados por locutor.

A análise final, tal como na app Streamlit, é feita pelo Claude via IAedu, a partir da transcrição diarizada.

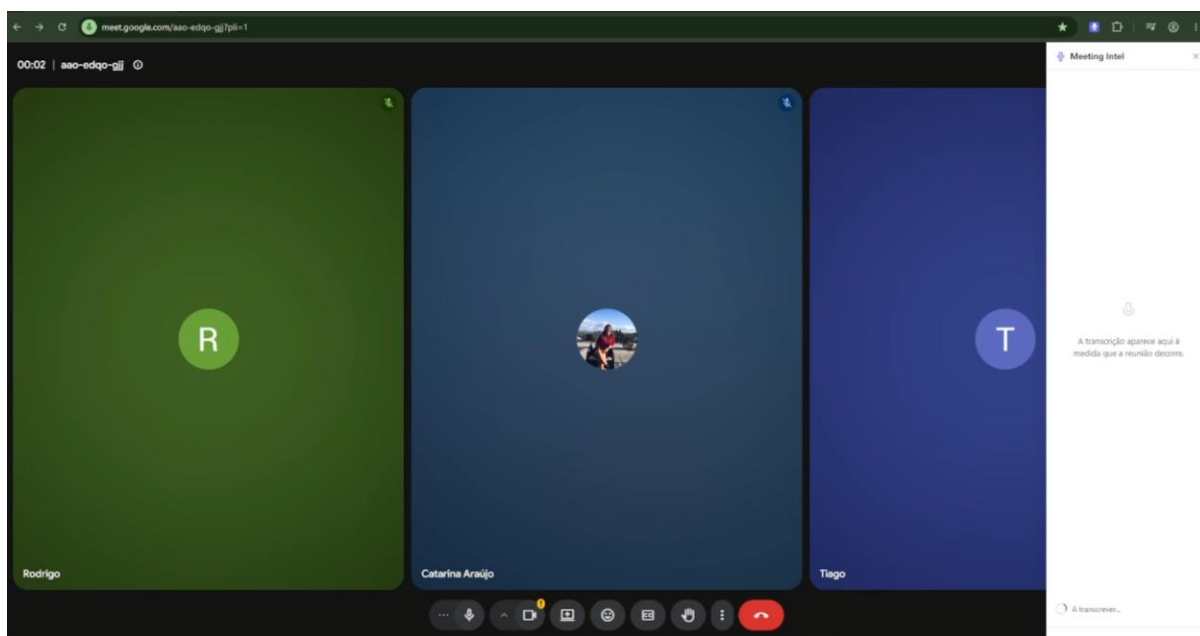


Figura 8 - Sidebar da extensão durante a captura de áudio

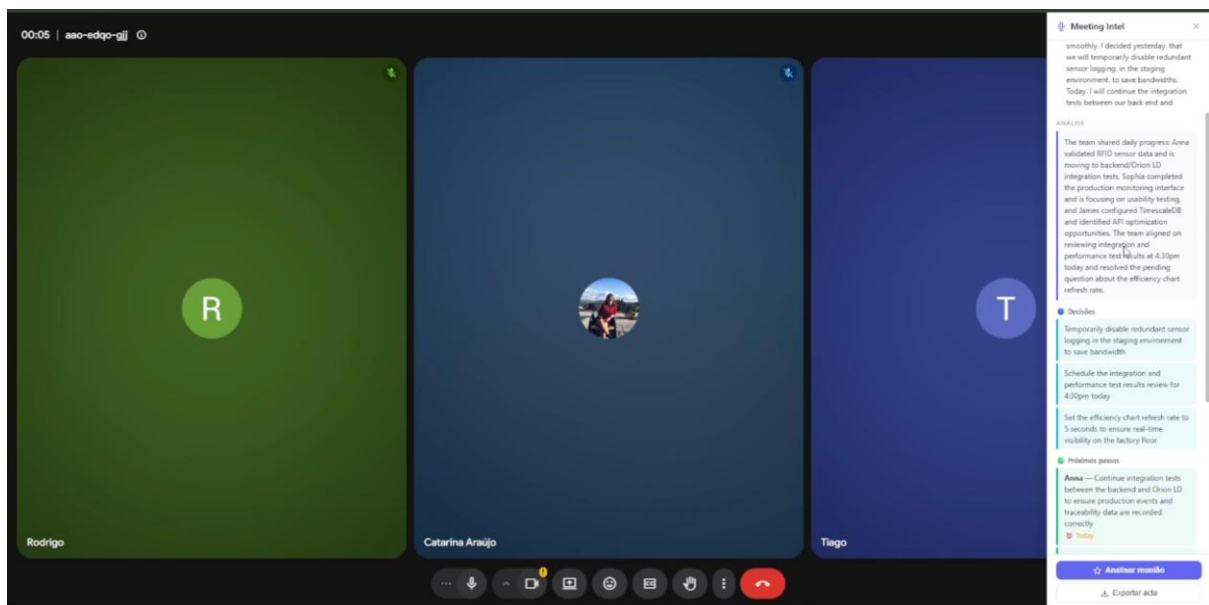


Figura 9 - Transcrição diarizada e análise estruturada apresentadas na sidebar, com cores distintas por locutor

3. Resultados e Validação

A validação do sistema foi feita através de **testes manuais**, com gravações de reuniões reais e simuladas, em ambas as componentes do projeto.

Na app Streamlit, foram testados ficheiros de áudio com múltiplos locutores, confirmando que o pipeline completo produz output coerente e que a exportação em TXT, Markdown e JSON funciona corretamente. O mecanismo de fallback entre transcrição via IAedu e Whisper local foi também verificado, acionando-se corretamente quando a API externa não aceita o ficheiro de áudio.

Na extensão Chrome, foram realizados testes em reuniões simuladas no Google Meet, com diferentes números de participantes. Foi confirmado que a captura mista de áudio do separador e do microfone funciona de forma consistente, que o servidor local processa corretamente os ficheiros recebidos, e que a sidebar apresenta a transcrição organizada por locutor seguida da análise gerada pelo Claude.

Não foram recolhidas métricas quantitativas formais, mas a observação direta durante os testes permitiu identificar pontos relevantes. A **qualidade da diarização** é fortemente dependente da qualidade do áudio recebido: o áudio proveniente do

WebRTC do Google Meet é comprimido e normalizado de forma semelhante entre todos os participantes, o que dificulta a distinção de vozes pelo pyannote. Ainda assim, ao misturar o stream do separador com o stream do microfone local, o pyannote consegue distinguir consistentemente o utilizador dos restantes participantes na maioria dos casos testados.

A **transcrição com Whisper** apresentou boa qualidade geral, tanto em português como em inglês, sem necessidade de especificar a língua manualmente, tendo sido identificado que forçar o parâmetro de língua provoca alucinações em segmentos de silêncio.

A **análise feita pelo Claude** mostrou-se consistentemente capaz de identificar corretamente decisões, próximos passos com responsável atribuído e questões em aberto, mesmo em transcrições com erros pontuais de reconhecimento de fala.

4. Dificuldades, Resolução e Limitações

Ao longo do desenvolvimento foram identificados e resolvidos vários problemas técnicos específicos do ambiente de extensões Chrome e do processamento de áudio.

4.1. Transcrição em tempo real inviável

A primeira versão da extensão tentava transcrever a reunião em tempo real, processando blocos de áudio de 8 em 8 segundos. Esta abordagem revelou-se inviável: o pyannote não mantém memória entre chamadas independentes, pelo que o **SPEAKER_00** identificado num bloco não correspondia necessariamente à mesma pessoa identificada como **SPEAKER_00** no bloco seguinte. A solução adotada foi **gravar o áudio completo da reunião continuamente e só correr a diarização e transcrição uma única vez**, no momento em que o utilizador termina a gravação.

4.2. Restrições de CORS

O content script injetado na página do Google Meet não tem permissão para fazer pedidos diretos a APIs externas. A solução foi centralizar todos os pedidos de rede no **service worker da extensão** (background.js), que não está sujeito às mesmas restrições.

4.3. Captura de áudio em Manifest V3

A API tradicionalmente usada para captar áudio de um separador foi descontinuada em Manifest V3. A solução final foi usar **getDisplayMedia()** diretamente no content script.

4.4. Conflito de cuDNN entre pyannote e Whisper

Ao correr ambos os modelos em GPU, surgia um erro de incompatibilidade de versões. A solução foi **forçar o pyannote a correr exclusivamente em CPU**, mantendo apenas o Whisper em GPU quando disponível.

4.5. Conflito entre bibliotecas de áudio

A biblioteca inicialmente usada para carregar os segmentos de áudio entrava em conflito com uma dependência interna do pyannote. A solução foi substituir essa biblioteca por **soundfile**.

4.6. Eco no microfone

O microfone tende a captar também o som proveniente dos altifalantes do utilizador. A ativação de cancelamento de eco e supressão de ruído mitiga parcialmente este problema, embora não o elimine por completo.

4.7. Outras Dificuldades

Apesar das soluções encontradas, ficaram por resolver algumas **limitações**. A **identificação de locutores pelo nome real** não foi implementada, já que o DOM do Google Meet apresenta os nomes de forma ofuscada, e os locutores são identificados apenas por labels genéricos. A **cancelação de eco no microfone**, apesar de ativa, não elimina por completo a captação do som dos altifalantes. A **transcrição em tempo real**, inicialmente prevista, não foi mantida devido à inconsistência dos labels de locutor entre chamadas independentes do pyannote, uma limitação inerente ao próprio modelo de diarização. Por fim, a **versão local** depende de GPU para correr com desempenho aceitável, ao contrário da versão API, acessível a qualquer pessoa.

5. Conclusão

O desenvolvimento do Meeting Intelligence permitiu validar, na prática, uma arquitetura de extração estruturada de informação a partir de áudio, combinando três modelos com responsabilidades bem definidas: o pyannote para diarização, o Whisper para transcrição e o Claude para análise semântica. Mais do que produzir um resumo de texto livre, o sistema demonstra que é possível obter dados estruturados e diretamente acionáveis (decisões, responsáveis, prazos) a partir de uma conversa não estruturada, cumprindo o objetivo central do projeto: eliminar a necessidade de tomar notas manualmente durante uma reunião.

O processo de desenvolvimento revelou também que grande parte da dificuldade não estava na escolha dos modelos em si, mas na integração entre eles e nas restrições impostas pelo ambiente de execução, em particular as limitações da API Manifest V3 do Chrome, os conflitos de dependências entre bibliotecas de áudio, e a ausência de memória entre chamadas independentes do pyannote, que obrigou a repensar a abordagem inicial de transcrição em tempo real. Estas dificuldades foram resolvidas através de soluções pragmáticas, como a centralização de pedidos de rede no service worker da extensão, a separação de modelos por dispositivo (CPU/GPU) para evitar conflitos, e a gravação completa da reunião antes de processar, em vez de processamento contínuo por blocos.

A existência de **duas versões da app Streamlit**, uma acessível a qualquer pessoa através de API keys e outra totalmente local, e de uma **extensão Chrome** dedicada a reuniões em tempo real, mostra que a mesma arquitetura conceptual pode adaptar-se a diferentes contextos de utilização e diferentes níveis de acesso a recursos computacionais. As limitações identificadas, sobretudo a identificação de locutores pelo nome real e a qualidade da diarização em áudio comprimido por WebRTC, definem as direções mais claras para trabalho futuro.

6. Bibliografia

Anthropic. (2024). *Claude documentation*. <https://docs.claude.com>

Anthropic. (2024). *Claude documentation*. <https://docs.claude.com>

Bredin, H. (2023a). *pyannote/speaker-diarization-3.1* [Model]. Hugging Face. <https://huggingface.co/pyannote/speaker-diarization-3.1>

Bredin, H. (2023b). *pyannote/segmentation-3.0* [Model]. Hugging Face. <https://huggingface.co/pyannote/segmentation-3.0>

Google. (2024). *Chrome extensions: Manifest V3 overview*. <https://developer.chrome.com/docs/extensions/develop/migrate/what-is-mv3>

IAedu. (2024). *IAedu — Claude API*. <https://chat.iaedu.pt/>

Pallets Projects. (2024). *Flask documentation*. <https://flask.palletsprojects.com/en/stable/>

OpenAI. (2022). *Whisper*. <https://openai.com/pt-PT/index/whisper/>

Streamlit Inc. (2024). *Streamlit documentation*. <https://docs.streamlit.io>

Araujo2112. (2026). *2026-ei-aoopii-c25* [Source code]. GitHub. <https://github.com/Araujo2112/2026-ei-aoopii-c25>